

前文讲解了负载均衡的全景图，包含网络，RPC，存储等。涉及技术面广，架构复杂。这些不同种类的负载均衡如何实现？虽然各个模块的功能有别，但是使用公共的负载均衡算法。

理解负载均衡的本质：一个节点把流量分发给下游的多个节点，简称为从 1 到 N。
负载均衡算法实现分发策略。

1、负载均衡常用如下算法

- 随机法
 - 权重随机法
 - 哈希法
 - 一致性哈希法
 - 轮询法
 - 权重轮询法
- （分多篇讲解）

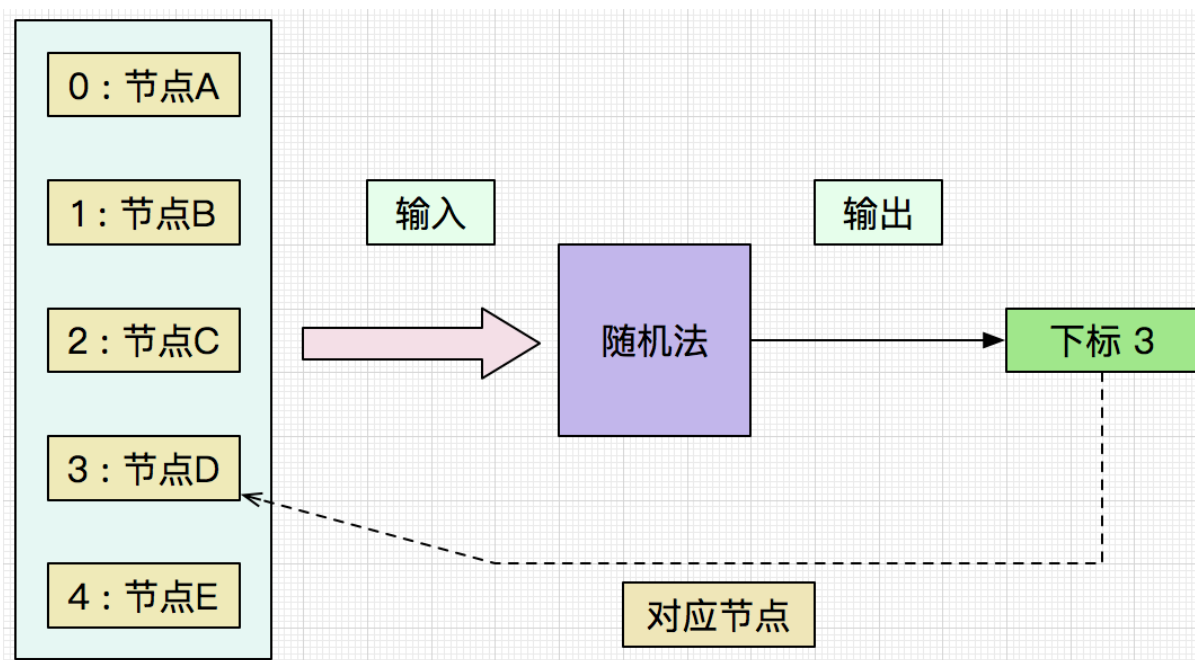
思维误区：大家可能认为，这些算法简单，作用不大。正好相反，极简主义方法论认为，简单意味着高效、易维护、易扩展，大道至简。
随着工作经验的增加，大家慢慢会有抽象的感悟：极简主义对于架构设计有根本性影响，系统越复杂，极简主义越重要。

2、随机法

随机法的含义：使用随机策略分发流量。

代码实现：借助 random 工具，获得一个随机的下标，用下标指向下个节点。

随机法是简单又高效的算法。



随机法的代码实现：

- 节点类

```
@Getter
@Setter
@Accessors(chain = true)
```

```
public class Node {
    public String server;// 节点信息
}
```

- 随机法函数。使用 Random+nodeList，即可快速实现。

```
// 随机法
public static Node balanceByRand(List<Node> nodeList) {
    int count = nodeList.size();
    Random rand = new Random();
    // 获得节点下标。
    int index = rand.nextInt(count);
    // 找到对应节点
    Node node = nodeList.get(index);
    return node;
}
```

- 准备一批节点

```
// 构建一批节点
Node nodeA = new Node().setServer("节点 A");
Node nodeB = new Node().setServer("节点 B");
Node nodeC = new Node().setServer("节点 C");
Node nodeD = new Node().setServer("节点 D");
Node nodeE = new Node().setServer("节点 E");
List<Node> nodeList = Arrays.asList(nodeA, nodeB, nodeC, nodeD, nodeE);
```

- 循环调用与统计

```
// 循环多次。
public static void loopRand(List<Node> nodeList, int loopCount) {
    System.out.println("\n loopCount = " + loopCount);
    // 计数
    Map<Node, AtomicInteger> countMap = new HashMap<>();
    for (int m = 0; m < loopCount; m++) {
        // 随机法
        Node chooseNode = balanceByRand(nodeList);
        // 计数
        AtomicInteger count = countMap.get(chooseNode);
        if (null == count) {
            count = new AtomicInteger(0);
            countMap.put(chooseNode, count);
        }
        count.incrementAndGet();
    }
    for (Map.Entry<Node, AtomicInteger> entry : countMap.entrySet()) {
        System.out.println(entry.getKey().server + " = " + entry.getValue().get());
    }
}
```

- 循环 1 万次，看每个节点的流量

```
// 循环 1 万次  
loopRand(nodeList, 10000);
```

```
loopCount = 10000  
节点 A = 1993  
节点 E = 1938  
节点 B = 2054  
节点 D = 1997  
节点 C = 2018
```

- 循环 100 万次，看每个节点的流量

```
// 循环 100 万次  
loopRand(nodeList, 1000000);
```

```
loopCount = 1000000  
节点 A = 200156  
节点 E = 199765  
节点 B = 199570  
节点 D = 200388  
节点 C = 200121
```

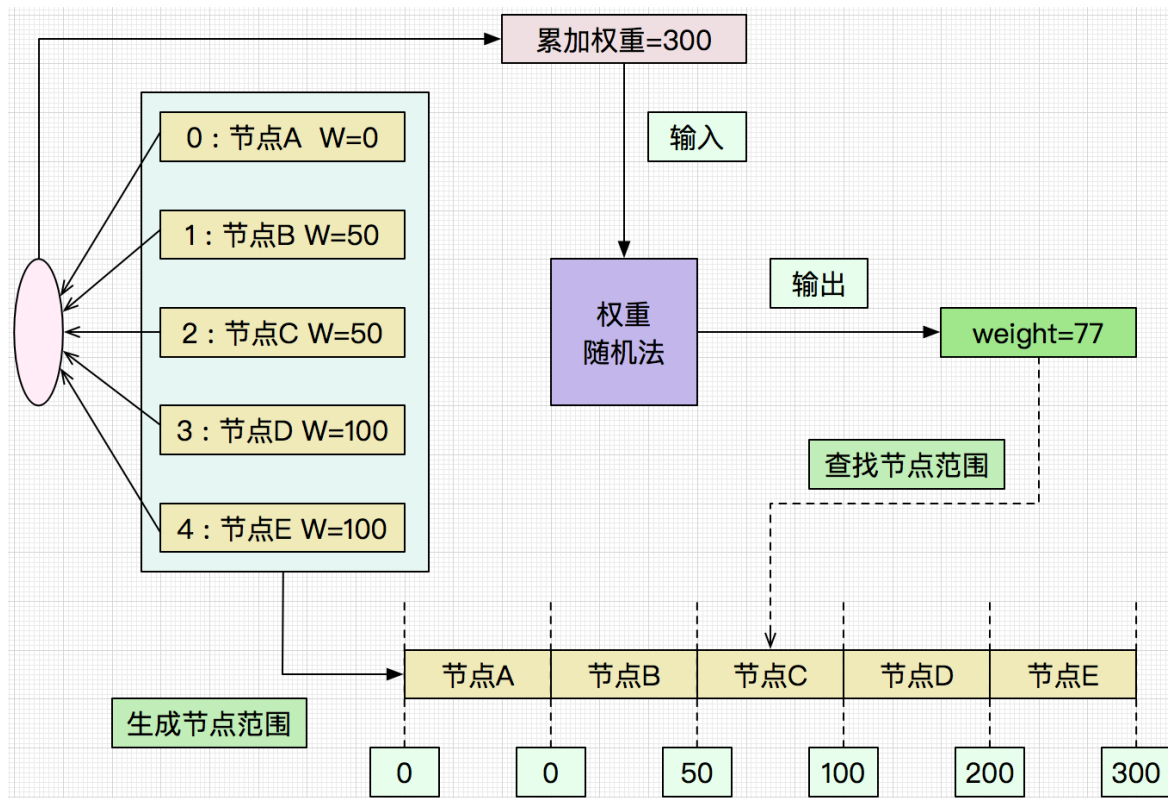
- 从测试结果看出，随机法使得流量均衡。
循环 1 万次，每个节点的流量约为 2000 。
循环 100 万次，每个节点的流量约为 20 万 。

3、权重随机法

权重随机法的含义：在随机法的基础上，给每个节点增加权重字段。

代码实现：把所有节点的权重累加起来，得到总权重，然后对总权重使用 `random` 获得随机权重，最后判断随机权重在哪个节点上。

权重随机法的特点在于，随机的对象不是节点下标，而是总权重。



为什么需要权重？

现实世界中，绝对平均不存在，物体之间有差异。在程序中，使用权重可以表示物体之间的差异。比如，某个东部省份的经济体量是某个西部省份的 N 倍，中央机房的计算能力是备份机房的 M 倍。

权重随机法的代码实现：

- 节点类

```
@Getter
@Setter
@Accessors(chain = true)
public class NodeWeight {
    public String server;// 节点信息
    public int weight;// 权重。从 0 到 100
    public int weightFrom;// 权重。下界
    public int weightTo;// 权重。上界
}
```

- 权重随机法函数。使用 Random+nodeList，即可快速实现。

```
// 权重随机法
public static NodeWeight balanceByRand(List<NodeWeight> nodeList) {
    // 累加总权重。同时，给每个 node 分配权重范围。
    int sumWeight = 0;
    for (NodeWeight node : nodeList) {
        node.setWeightFrom(sumWeight);
        sumWeight += node.weight;
        node.setWeightTo(sumWeight);
    }
}
```

```

// 获得随机权重。
Random rand = new Random();
int indexWeight = rand.nextInt(sumWeight);
for (NodeWeight node : nodeList) {
    // 找到随机权重对应的 node。
    if (node.weightFrom <= indexWeight && indexWeight < node.weightTo) {
        return node;
    }
}
return null;
}

```

- 准备一批节点

```

// 构建一批节点
NodeWeight nodeA = new NodeWeight().setServer("节点 A").setWeight(0);
NodeWeight nodeB = new NodeWeight().setServer("节点 B").setWeight(50);
NodeWeight nodeC = new NodeWeight().setServer("节点 C").setWeight(50);
NodeWeight nodeD = new NodeWeight().setServer("节点 D").setWeight(100);
NodeWeight nodeE = new NodeWeight().setServer("节点 E").setWeight(100);
List<NodeWeight> nodeList = Arrays.asList(nodeA, nodeB, nodeC, nodeD, nodeE);

```

- 循环调用与统计

```

// 循环多次。
public static void loopRand(List<NodeWeight> nodeList, int loopCount) {
    System.out.println("\n loopCount = " + loopCount);
    // 计数
    Map<NodeWeight, AtomicInteger> countMap = new HashMap<>();
    for (int m = 0; m < loopCount; m++) {
        // 随机法
        NodeWeight chooseNode = balanceByRand(nodeList);
        // 计数
        AtomicInteger count = countMap.get(chooseNode);
        if (null == count) {
            count = new AtomicInteger(0);
            countMap.put(chooseNode, count);
        }
        count.incrementAndGet();
    }
    for (NodeWeight node : nodeList) {
        AtomicInteger countRef = countMap.get(node);
        int count = null == countRef ? 0 : countRef.get();
        System.out.println(node.server + " = " + count);
    }
}

```

- 循环 1 万次，看每个节点的流量

```
// 循环 1 万次  
loopRand(nodeList, 10000);
```

```
loopCount = 10000  
节点 A = 0  
节点 B = 1688  
节点 C = 1694  
节点 D = 3345  
节点 E = 3273
```

- 循环 100 万次，看每个节点的流量

```
// 循环 100 万次  
loopRand(nodeList, 1000000);
```

```
loopCount = 1000000  
节点 A = 0  
节点 B = 167245  
节点 C = 166696  
节点 D = 333478  
节点 E = 332581
```

- 从测试结果看出，权重随机法使得流量对应权重。

5 个节点的权重分别为 0、50、50、100、100，获得权重比率 0:1:2。

循环 1 万次，A 为 0，B、C 为约 1690，D、E 为约 3300。与权重比率对应。

循环 100 万次，A 为 0，B、C 为约 167000，D、E 为约 333000。与权重比率对应。

权重随机法的设计关键：给 node 增加 weight 属性，累加总权重时给每个 node 分配权重范围 weightFrom/weightTo，后续获得随机 weight，反过来遍历权重范围找到对应的 node。